

Kalman Filter: Milestone-3

RISC-V Scalar Assembly Implementation

Deadline: 13th April

1 Purpose of this Milestone

Milestone 3 marks the transition from high-level C/C++ implementation to low-level RISC-V scalar assembly programming. Building directly on the LKF and EKF implementations from Milestone 2, students will re-implement the core computational functions of both filters in **RISC-V scalar assembly**.

This milestone develops a fundamental understanding of how abstract mathematical operations — matrix multiplications, LU decompositions, Jacobian evaluations — map onto actual processor instructions, register files, memory hierarchies, and calling conventions.

2 Background: RISC-V Architecture Recap

2.1 Registers

RISC-V provides 32 integer registers (**x0–x31**) and 32 floating-point registers (**f0–f31**). Table 1 summarises the names and their roles.

Table 1: RISC-V ABI Register Summary (selection)

ABI Name	Register	Role
zero	x0	Hardwired 0
ra	x1	Return address
sp	x2	Stack pointer
a0--a7	x10--x17	Arg / return
s0--s11	x8--x9, x18--x27	Saved regs
t0--t6	x5--x7, x28--x31	Temporaries
fa0--fa7	f10--f17	FP arg / return
fs0--fs11	f8--f9, f18--f27	FP saved
ft0--ft11	f0--f7, f28--f31	FP temporaries

2.2 Relevant Instruction Classes

Students must be comfortable with the following instruction subsets:

- **Integer arithmetic:** `addi`, `add`, `sub`, `mul`, `slli`, `srli`, `srai`.
- **Integer load/store:** `ld`, `sd`, `lw`, `sw`.
- **FP double arithmetic:** `fadd.d`, `fsub.d`, `fmul.d`, `fdiv.d`, `fsqrt.d`.
- **Fused multiply-add:** `fmadd.d`, `fmsub.d`, `fnmsub.d` — students must use these whenever a multiply-accumulate pattern appears.
- **FP load/store:** `fld`, `fsd`.
- **FP compare:** `flt.d`, `fle.d`, `feq.d`.
- **FP/int conversion:** `fcvt.d.w`, `fcvt.w.d`.
- **Control flow:** `beq`, `bne`, `blt`, `bge`, `jal`, `jalr`.

3 RISC-V Setup

Use the RISC-V setup guide provided separately.

4 Problem Description

Using the same 23-joint full-body gait dataset from Milestone 2, students must re-implement the following computational components entirely in RISC-V scalar assembly:

1. Core equations of LKF and EKF.
2. Matrix Inverse technique (same as used in Milestone-2).
3. Algorithm used to calculate arctan.
4. Memory management (stack/heap).

5 State and System Matrices

The state vector and all system matrices ($\mathbf{F}$, $\mathbf{Q}$, $\mathbf{H}$, $\mathbf{R}$) are identical to Milestone 2. For reference:

$$N = 12 \times 23 = 276, \quad M = 3 \times 23 = 69 \quad (1)$$

$$\mathbf{F} \in \mathbb{R}^{276 \times 276}, \quad \mathbf{P} \in \mathbb{R}^{276 \times 276}, \quad \mathbf{H} \in \mathbb{R}^{69 \times 276}, \quad \mathbf{R} \in \mathbb{R}^{69 \times 69} \quad (2)$$

6 Numerical Verification

Assembly output must be verified against the Milestone 2 C/C++ reference. For each frame k and each state component i , the absolute error must satisfy:

$$\left| \hat{x}_{k,i}^{\text{asm}} - \hat{x}_{k,i}^{\text{C/C++}} \right| \leq \epsilon_{\text{tol}} = 10^{-9} \quad (3)$$

Students must produce a verification table in their report reporting average error across state components for each joint and average error across joints for each state component. Also report maximum and minimum error.

7 Implementation Requirements

- Implement two separate assembly files: `lkf_asm.s` (LKF) and `ekf_asm.s` (EKF).
- A `Makefile` must be provided that compiles and run all `.s` files.
- Push all work to a new branch named Milestone-3.
- estimated data output files to be uploaded on github.
- plots (same as in milestone-2) required for the estimated data from milestone-3.
- comparison plots for outputs from milestone-2 and milestone-3.

8 Required Analysis and Report Content

The latex-format PDF report must include:

1. **Function-by-function walkthrough:** for each assembly functions, a code snippet of the function along with its explanation.
2. **Register allocation:** for each function and matrices, how and which registers have been used.
3. **Numerical verification table** as described in Section 6.
4. **Plots** as described in section 7.

9 GitHub Repository Requirements

Push all work to branch **milestone-3** of the existing repository. The branch must contain:

- All `.s` assembly source files.
- `Makefile`.
- estimated output files for LKF and EKF.

10 Evaluation Rubric (100 Points)

Component	Points
Linear Kalman Filter (LKF) Implementation	20
Extended Kalman Filter (EKF) Implementation	20
Matrix Inversion	10
Memory Management and register allocation	10
<code>arctan2</code> Approximation (EKF)	10
Plots and analysis	10
Estimated data output files	5
Numerical Verification against Milestone-2	5
Report Quality and Code Explanation	10
Total	100

Table 2: Milestone-3 Evaluation Rubric